LMA Major Project Final Report

1. Introduction

In an era of complex financial products and economic uncertainty, achieving personal financial stability requires more than just information—it requires clear, reliable, and actionable guidance. This project addresses that need by designing and implementing a sophisticated **Financial Literacy Subject-Matter Expert (SME) AI Agent**, a highly extensible Retrieval-Augmented Generation (RAG) system. The agent is engineered to serve as a specialized authority in personal finance, moving beyond simple definitions to perform complex, multi-step tasks with deep reasoning and autonomous tool-use.

The core objective of this SME Agent is to understand a user's complex financial request, formulate a reasoned plan of action, and execute it by orchestrating a series of specialized tools. This includes retrieving relevant principles from a curated knowledge base of financial literature, generating high-quality and personalized content (such as customized budget plans, savings strategies, or investment explanations), exporting them into practical formats like PDF budgets and automating their delivery via email. A fundamental capability is robust, interactive Q&A, where the agent grounds its advice in sound financial principles to ensure accuracy and prevent misinformation—crucial in a domain where decisions have real-world consequences.

Built with a modular and scalable architecture, the system incorporates advanced data preprocessing and is fine-tuned on a high-quality corpus of financial documents. This allows it to master the nuances of concepts like compound interest, debt management, and risk diversification. With integrated self-learning mechanisms that adapt based on user feedback, the agent continuously refines its ability to provide clear, empowering, and personalized financial guidance.

This project delivers not just a functional prototype, but a comprehensive framework for a financial assistant that is truly expert, trustworthy, and capable of automating the creation of tailored financial plans to promote economic well-being.

2. Dataset

This study utilizes a comprehensive dataset comprising over 100 PDF documents focused on finance, banking, and financial literacy with particular emphasis on the Indian financial landscape. The dataset encompasses a diverse range of materials including foundational financial education resources, regulatory certification workbooks from organizations such as SEBI and NISM, academic research on financial behavior and literacy, industry reports on banking and securities markets, investment guides covering stocks, real estate, and gold, and core financial and accounting textbooks. The collection covers critical topics such as personal budgeting, mutual funds, derivatives trading, monetary policy, taxation, risk management, and anti-money laundering provisions, making it a valuable resource for comprehensive analysis of financial education, investment patterns, and market dynamics in India. By combining regulatory materials, academic research, and industry insights, this dataset provides a multifaceted perspective on financial systems, investment opportunities, and financial literacy initiatives in the Indian context.

Dataset: https://huggingface.co/datasets/Karan007/financial-literacy-lma

2.1 RAG Pipeline

The complete RAG pipeline consists of four primary stages: data extraction, data cleaning and preprocessing, multi-granularity embedding generation with indexing, and query retrieval with reranking. Each stage is described below in detail.

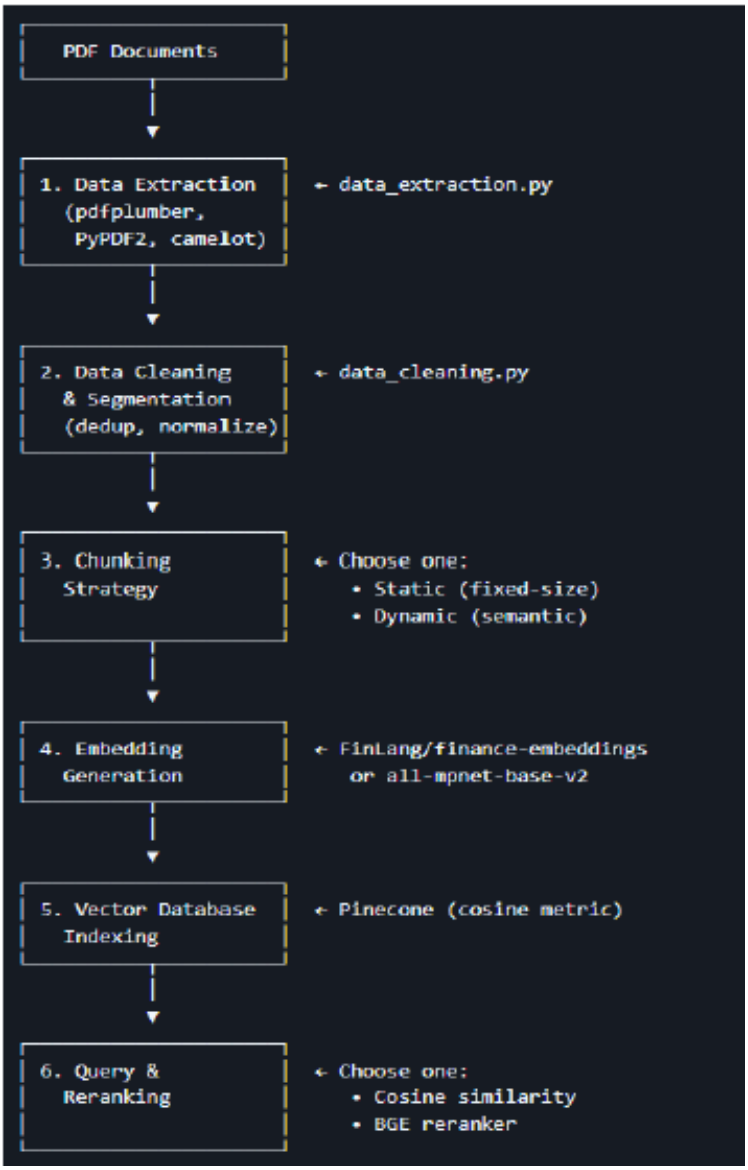
Data Cleaning and Segmentation

The data cleaning phase was implemented using the script 'data_cleaning.py'. Each document underwent cleaning to remove duplicate lines, non-breaking spaces, and formatting artifacts. The text was normalized to lowercase and tokenized using NLTK’s sentence tokenizer. The processed data was stored as 'knowledge_base_cleaned_segmented.jsonl'.

Multi-Granularity Embedding and Indexing

The 'data_embedding.py' script was responsible for embedding generation and vector indexing. We used the **'FinLang/finance-embeddings-investopedia'** model from Sentence Transformers for domain-specific embeddings. Each cleaned text segment was split into multiple granularities of 128, 512, and 2048 tokens, with a 50-token overlap to preserve context continuity. These embeddings (dimension 768) were stored in a Pinecone vector

database using cosine similarity as the distance metric. Metadata such as document ID, page number, and granularity level were retained for interpretability.



Query Retrieval and Reranking

The 'reranking.py' module handles user queries and implements semantic retrieval. When a query is issued, its embedding is computed using the same 'FinLang' model. An initial candidate set (approximately 7× the top-k) is fetched from the Pinecone index. To improve relevance, the system employs a reranking mechanism using **'BAAI/bge-reranker-v2-m3'** from the FlagEmbedding library, which reorders the results based on semantic similarity. This ensures that the most contextually relevant chunks are prioritized.

Chunking Strategy

The main function responsible for chunking is segment_text_hybrid(text). It operates by first structuring the text based on its natural document hierarchy and then applying semantic grouping, followed by fixed-size token-based splitting.

Fixed Chunking strategy using Independent multi granularity chunks

Fixed chunking strategy divides text into equal-sized segments based on a set number of tokens, words, or characters, without considering sentence meaning. It ensures uniform chunk sizes, making it simple and efficient for embedding and indexing. However, it may split sentences or lose context, so small overlaps are often added between chunks to preserve continuity.

Document/Paragraph Splitting and Semantic chunking

The raw text is first split into paragraphs using newline characters (\n).

This ensures that the chunking process respects the original document structure, preventing chunks from spanning across distinct paragraphs that might discuss different topics.

Each paragraph is further divided into sentences using nltk.tokenize.sent_tokenize.

The semantic_chunk function then groups consecutive sentences. A new "semantic chunk" is started if the cosine similarity between the embeddings of the current sentence and the previous one drops below a defined SEMANTIC_THRESHOLD (0.55).

This aims to keep sentences that are topically cohesive together, as a drop in semantic similarity often indicates a topic shift.The final, largest step is to take the semantically-grouped chunks and split them into smaller, fixed-size chunks based on token count and store in parent-child relationship.

A crucial feature is the overlap mechanism: consecutive child chunks overlap by OVERLAP_TOKENS (50 tokens).

Purpose of Overlap: This preserves context that might otherwise be cut off at the boundaries, ensuring that key information or sentence continuity isn't lost between chunks.

In this project, a parent-child chunking strategy was used with chunk sizes of 128, 512, and 2048 tokens, maintaining a 50-token overlap between consecutive chunks. This approach was implemented to handle the heterogeneous nature of financial documents, which range from short definitions to long regulatory explanations.

Cosine Similarity

Cosine similarity is a measure used in vector databases like Pinecone to determine how similar two embedding vectors are in terms of their direction rather than magnitude. It calculates the cosine of the angle between two vectors. For example, the embeddings of “Emergency fund helps in crisis” and “Savings for unexpected events” would have a cosine similarity close to 1, indicating they represent similar financial concepts, whereas unrelated sentences like “Invest in long-term stocks” would have a much lower similarity score.

After initial retrieval, the top candidates are reranked using the BGE Reranker (BAAI/bge-reranker-v2-m3) from the FlagEmbedding library.

Why Use a Reranker?

The first retrieval (using Pinecone) is purely vector-similarity-based, which captures surface-level semantic closeness but can miss nuanced contextual relationships.

The reranker computes a contextual relevance score between the query and each retrieved chunk — ensuring results that semantically answer the question are prioritized.

2.2 Example Retrieval Result

Experiment 1 (Fixed chunking) :An example query was tested to evaluate the RAG system’s performance. The query used was: 'What is an emergency fund?'. Below are the top retrieved and reranked results:

Rank	Source	Initial Score	Excerpt
1	NISM Series X-A Investment Adviser Level 1.pdf	0.5315	The fund should be adequate to meet the expenses for six months...
2	Financial-Literacy-Basics-for-Adults.pdf	0.4928	Step 4: Emergency fund building and

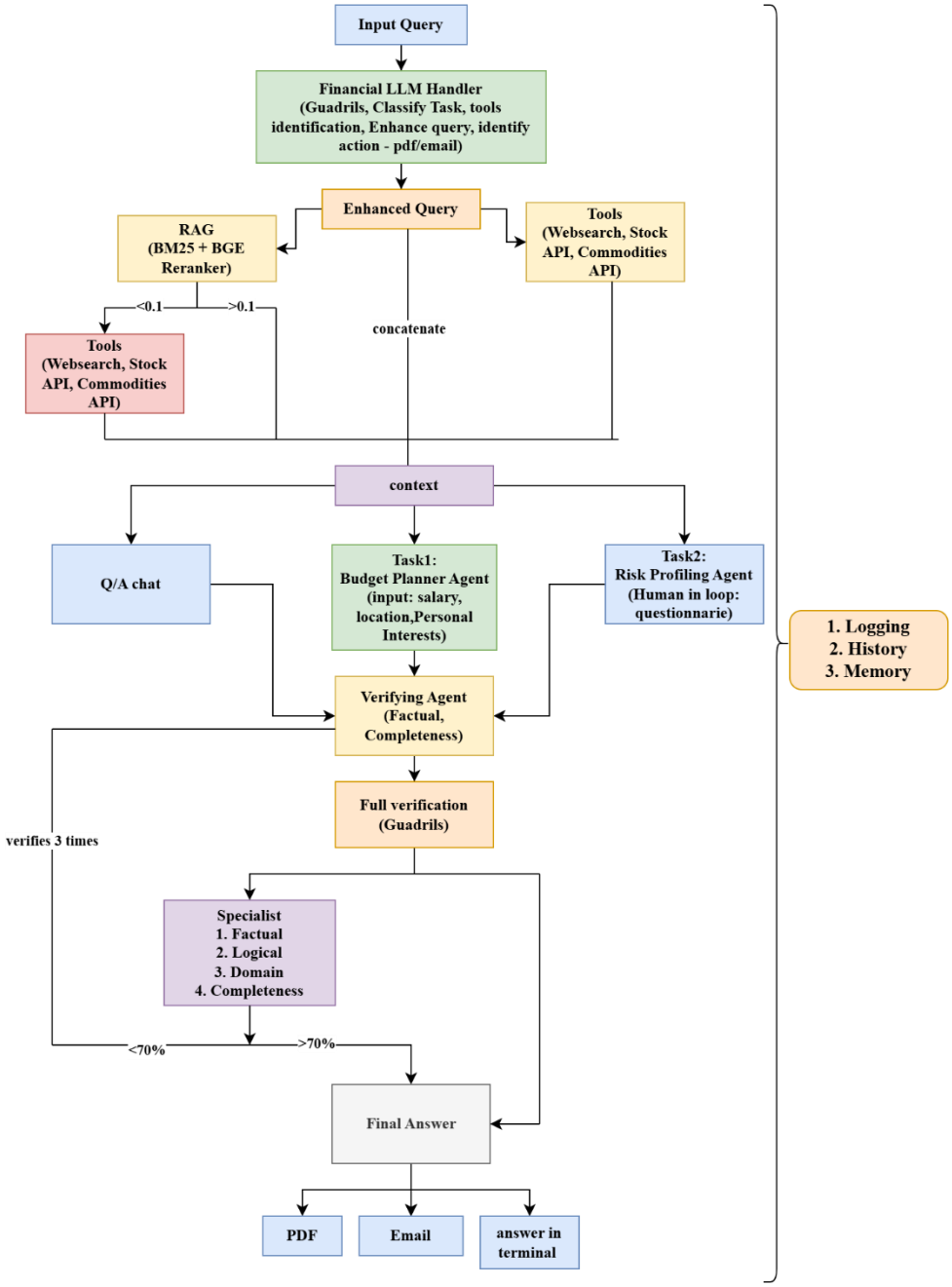
			maintaining an emergency fund...
3	NISM Series X-A Investment Adviser Level 1.pdf	0.4868	If the emergency fund is used then efforts must be made to replenish it...

Experiment 2 (Dynamic chunking): An example query was tested to evaluate the RAG system's performance. The query used was: 'What is an emergency fund?'. Below are the top retrieved and reranked results:

Rank	**Source**	**Initial Score**	**Excerpt**
1	book3.pdf	0.4239	an etf is a basket of securities that can trade like stocks on a stock exchange.
2	Deutsche-Bank-Namaste-India-2025.pdf	0.4072	'specified fund' is defined in the explanation to section 10(4d) of the it act.
3	book3.pdf	0.2976	this market is referred to as the market for loanable funds. in the market for loanable funds, the suppliers of funds are economic entities that currently..
4	Deutsche-Bank-Namaste-India-2025.pdf	0.2971	category i aif: invests in start-up or early-stage ventures or social ventures or smes or infrastructure or other sectors or areas which the government or regulators consider as socially or economically desirable and shall include venture capital funds, sme funds, social impact funds, …

5	book3.pdf	0.2756	federal reserve (fed) targets the equilibrium interest rate on federal funds as one of its most important monetary policy tools. the federal funds market traditionally consists of the overnight …

3. Architecture and Pipeline



3.1 Components

Component	Expanded Role and Function
Task Detector	**Intelligent Intent Classification & Complexity Analysis:** This is the initial filter, responsible for correctly understanding the user's

Component	Expanded Role and Function
	intent. It employs a specialized Machine Learning model, such as **FinBERT**, which is pre-trained on financial text, making it highly accurate at classifying financial queries into defined categories (e.g., BUDGET_PLANNING, RISK_PROFILING, GENERAL_QA). Beyond classification, it performs **Complexity Analysis** (checking query length, number of variables, and calculation keywords) to prepare the Orchestrator for the required depth of processing.
Orchestrator	**Central Command, Routing, and State Management:** The Orchestrator is the mission control. It receives the intent from the Task Detector and acts as a sophisticated **State Machine**. It is responsible for: 1) **Routing** the query to the correct Specialized Task Handler. 2) **Managing the entire lifecycle** of the answer generation. 3) **Controlling the Multi-Agent Verification Loop** (determining when to start a verification check, which Specialist to use, and when the process is complete).
Specialized Task Handlers	**Domain Execution and Contextualization:** These are self-contained modules for handling specific, complex financial workflows. They transform a natural language request into a structured action. For instance, the **Budget Planner Handler** doesn't just ask an LLM for a budget; it applies internal, pre-coded financial rules (like the 50/30/20 principle), collects necessary user context (income, location), and formats the output into a structured data object ready for reporting. The **Risk Profiler Handler** specifically manages the questionnaire and calculation of an asset allocation based on risk tolerance.
Specialist Agents	**Automated Quality Assurance Team:** This is a team of distinct, narrowly-focused AI agents whose sole purpose is to audit and improve the initial answer. They prevent common LLM pitfalls by checking for their assigned error type: **Data Specialist** (factual

Component	Expanded Role and Function
	verification against RAG/Data), **Logic Specialist** (mathematical correctness and procedural steps), and **Compliance Specialist** (adherence to predefined regulatory rules, such as mandatory emergency fund minimums).
RAG/Data Providers	**External Knowledge and Real-Time Grounding:** This layer is crucial for preventing *hallucination*. The **Retrieval-Augmented Generation (RAG)** system queries proprietary or high-quality financial knowledge bases to ground the LLM's response with verified documents. **Live Data Providers** pull current market data (e.g., stock prices, interest rates), ensuring the advice is based on the absolute latest figures.
Output Generators	**Professionalization and Actionable Delivery:** These tools bridge the gap between AI generation and professional client communication. The **PDF Generator** (using tools like ReportLab) takes the verified structured data and converts it into a clean, formal document with professional formatting, tables, and branding. The **Email Sender** provides a secure method to deliver these reports, completing the user experience with a polished, actionable deliverable.

3.2 Workflow

Step 1: Query Ingestion and Intelligent Classification

- **User Input:** The process begins when the user submits their financial query (e.g., "Calculate my budget and send it as a PDF").
- **Task Detector Activation:** The query is immediately sent to the **Task Detector**. This component doesn't rely on simple keyword matching; it uses a sophisticated **FinBERT** model to determine the user's underlying goal and intent.

- **Contextual Analysis:** Simultaneously, the Detector analyzes the query's complexity and extracts key variables (like location, income, or investment timeframe). The result is a highly structured request packet passed to the Orchestrator.

Step 2: Initial Answer Generation (The First Draft)

- **Routing:** The **Orchestrator** uses the classified intent to route the request to the correct **Specialized Task Handler** (e.g., the Budget Planner Handler).
- **Contextual Drafting:** The handler then constructs a detailed, contextualized prompt for the underlying Language Model, injecting the user's specific variables and any necessary information retrieved from the **RAG/Data Providers**.
- **Initial Output:** The Language Model generates the **Initial Answer**—the first draft of the advice or plan.

Step 3: The Multi-Agent Verification Loop (Quality Control)

This is the most critical phase, designed to fix three core types of LLM errors: factual inaccuracy, logical failure, and compliance violation.

1. **Issue Detection:** The Orchestrator's internal **Issue Detector** scans the Initial Answer against a baseline of common errors. This step quickly determines which of the **Specialist Agents** needs to be activated first.
2. **Specialist Intervention:** Based on the detected issue, the Orchestrator assigns the task to the most relevant specialist:
    - **Data Specialist:** Compares every fact, figure, and claim in the answer against live or RAG-sourced external data. It ensures, for example, that the referenced tax bracket or stock price is current and verifiable.
    - **Logic Specialist:** Analyzes the logical flow and arithmetic. If the advice involves a calculation (like compounding interest or calculating a debt ratio), this specialist verifies the mathematical steps and final result.
    - **Compliance Specialist:** This agent holds a defined set of **financial regulatory rules** (e.g., "A client must have a 3-6 month emergency fund before investing"). It checks the advice exclusively against these rules, ensuring the guidance is safe and responsible.

3. **Iterative Refinement:** If a Specialist Agent finds an error, it generates a **Correction Prompt** and uses the LLM to produce a **Corrected Answer**. This corrected answer is immediately fed back into the beginning of the Verification Loop to be re-checked by all specialists, ensuring a comprehensive, multi-layered quality check before moving forward.

Step 4: Final Output and Professional Delivery

- **Final Acceptance:** Once the answer passes all specialist checks (or hits the maximum iteration limit), the **Orchestrator** accepts the final, high-confidence output.
- **Response Assembly:** The final advice is packaged into an **Agent Response** object, complete with metadata documenting the task type and the entire verification log.
- **Deliverable Generation:** If the original query required a file (e.g., "Give me the budget planner in PDF format"), the **Output Generators** are activated. The **PDF Generator** converts the structured financial data into a professional document, and the **Email Sender** delivers the final report securely to the user, finalizing the end-to-end professional service.

3.3 Features

1. Multi-Agent System

- **DataSpecialist**: Fixes factual errors by re-retrieving data
- **LogicSpecialist**: Fixes mathematical and logical errors
- **ComplianceSpecialist**: Fixes regulatory and compliance issues
- **CompletionSpecialist**: Expands incomplete answers

2. Dynamic Verification Loop

- Runs up to **3 iterations** of verification
- **5 types of issue detection**:
    i. Factual Accuracy (vs knowledge base)
    ii. Logical Consistency (math & calculations)
    iii. Compliance (financial rules & regulations)
    iv. Completeness (sufficient information)
    v. Source Attribution (citations)
- **Automatic routing** to appropriate specialist
- **Confidence tracking** through iterations
- **Self-healing**: Automatically fixes errors

3. Advanced RAG Pipeline

- **Vector Retrieval**: Using semantic embeddings
- **BM25 Search**: Keyword-based retrieval
- **Hybrid Ranking**: Combines both methods
- **Score Normalization**: Proper ranking
- **Async Operations**: Non-blocking I/O

4. Multiple LLM Support

- **Primary**: Google Gemini-2.5-Flash
- **Fallback 1**: Ollama (local)
- **Fallback 2**: HuggingFace
- **Retry Logic**: Automatic retries with backoff
- **Async Generation**: Non-blocking calls

5. Production Quality

- **Comprehensive Logging**: All operations logged
- **Error Handling**: Graceful degradation
- **Type Hints**: Full type safety
- **Async/Await**: Throughout entire codebase
- **Configuration Management**: Centralized settings
- **Data Classes**: Structured data with validation

6. Full Transparency

- **Reasoning Chains**: See every decision step
- **Iteration Logs**: Track all verification iterations
- **Specialist Usage**: Know which agents were used
- **Confidence Scores**: Understand answer reliability
- **Source Attribution**: See where information came from

3.4 API's

1. Finnhub (Stocks & Commodities)

FINNHUB_API_KEY = "d4c5ct9r01qoua32jb9gd4c5ct9r01qoua32jba0" # Get from https://finnhub.io

FINNHUB_BASE_URL = https://finnhub.io/api/v1

2. Alpha Vantage (Stocks & Forex)

ALPHA_VANTAGE_API_KEY = "8F5PNHOG03L5A3RQ" # Get from https://www.alphavantage.co

ALPHA_VANTAGE_BASE_URL = https://www.alphavantage.co/query

3. Numbeo (Cost of Living)

NUMBEO_API_KEY = "YOUR_NUMBEO_KEY" # Get from https://www.numbeo.com/api/

NUMBEO_BASE_URL = https://www.numbeo.com/api/v2

4. News API (Web Search Alternative)

NEWS_API_KEY = "9e46aebf2cf54bc1aef10e141995dd3e"

Get from https://newsapi.org

5. Serpstack (Web Search - requires subscription)

SERPSTACK_API_KEY = "de080a3d694e86cc9a524bc9bbe060a1"

Get from https://serpstack.com

4. Prompt Engineering

4.1 Q/A chat Prompt

```
 prompt =f"""You are a professional financial advisor providing personalized guidance on finance.

        CHAT HISTORY:
        {self._load_chat_history(user_id)}

        CONTEXT FROM PINECONE HYBRID RETRIEVAL:
        {rag_context[:1000] if rag_context else 'No context available'}

        REQUIREMENTS:
        1. Be accurate and specific
        2. Include reasoning
        3. Mention relevant financial principles
        4. **CRITICAL:** If the CONTEXT provided is irrelevant to the USER QUESTION, you MUST ignore the context and answer based only on your general financial knowledge, without stating that the provided context was not useful for the current question.
        5. Include appropriate disclaimers if needed
        6. Be structured and easy to follow
```

```
        7 **OUTPUT STRUCTURE:** If User has asked about sending email, do not
say that as AI I can not send email. Just provide the JSON output. with email
body and context that user has asked for.
        if User has asked for PDF generation, do not say that as AI I can not
generate PDF. Just provide the Markdown output with PDF content that user has
asked for.

        USER QUESTION:
        {query}
        ---
        FINAL RESPONSE:

        If User has asked about sending email or generate PDF, do not say that
as AI I can not send email.
            - do not draft any email content. Just provide the Markdown output
with PDF content that user has asked for.
            - do not mention like this:
                Subject: Your Personalized Monthly Budget Plan
                Hi Karan,
                Here is the detailed monthly budget plan you requested,
designed to help
                you manage your 50000 income effectively and achieve your
financial goals.

            -ONLY draft PDF content that user has asked for like this.
                Monthly Budget Plan: 50000
                Essentials: 20000 (Rationale: Covers......
        If none of the above, provide the final answer in the requested
Markdown format.
        (Return ONLY the Markdown object)
        ---

        ANSWER:"""
```

4.2 Budget Planner prompt

```
"""Generate personalized budget plan"""

        prompt = f"""
        You are a certified, meticulous personal finance advisor.

        CHAT HISTORY:
        {self._load_chat_history(user_id)}

        Your MANDATORY Task:
        Generate a complete, strictly valid budget plan in the final requested
Markdown format.
```

```
        CONTEXT (Use for guidance, but prioritize Rules):
        {rag_context[:1000] if rag_context else 'No context available'}

        USER QUESTION/INPUT:
        {query}

        ---
        STRICT OUTPUT GENERATION RULES:
        ---
        1.  **MANDATORY CATEGORIES:** You MUST include ALL five of the
following categories in the final output structure:
            * Essentials
            * Savings
            * Discretionary
            * Debt Repayment
            * Insurance/Safety

        2.  **NUMERICAL CONSTRAINTS:**
            * The **Savings** allocation MUST be greater than or equal to 20%
of the total income (Savings >= 20% of Income).
            * **Essentials** MUST explicitly include allocations for: housing,
food, and utilities.

        3.  **RATIONALE:** You MUST provide a short, specific rationale
*immediately* following the allocated amount for each of the five mandatory
categories.

        4.  **FORMATTING/EXCLUSION:**
            * The ONLY output allowed is the final budget plan in the
requested Markdown format.
            * You MUST NOT include any introductory sentences, conversational
text, closing remarks, or commentary outside of the final structured output.

        5. **OUTPUT STRUCTURE:** If User has asked about sending email, do not
say that as AI I can not send email. Just provide the JSON output. with email
body and context that user has asked for.
        if User has asked for PDF generation, do not say that as AI I can not
generate PDF. Just provide the Markdown output with PDF content that user has
asked for.
        ---
        MANDATORY OUTPUT FORMAT TEMPLATE:
        (Use such structure for the final answer as given below and add more
details as much as possible, add table if needed)
        ---
        ## 💰 Monthly Budget Plan: [Insert Income Amount Here]

        * **Essentials:** [Numeric Value] (Rationale: [Insert Rationale Here])
```

```
            * Housing: [Numeric Value]
            * Food: [Numeric Value]
            * Utilities: [Numeric Value]
        * **Savings:** [Numeric Value] (Rationale: [Insert Rationale Here])
        * **Debt Repayment:** [Numeric Value] (Rationale: [Insert Rationale
Here])
        * **Insurance/Safety:** [Numeric Value] (Rationale: [Insert Rationale
Here])
        * **Discretionary:** [Numeric Value] (Rationale: [Insert Rationale
Here])
        ---
        **Total Allocated:** [Sum of all main category values]

        ---
        FINAL RESPONSE:
        If User has asked about sending email or generate PDF, do not say that
as AI I can not send email.
            - do not draft any email content. Just provide the Markdown output
with PDF content that user has asked for.
            - do not mention like this:
                Subject: Your Personalized Monthly Budget Plan
                Hi Karan,
                Here is the detailed monthly budget plan you requested,
designed to help
                you manage your 50000 income effectively and achieve your
financial goals.

            -ONLY draft PDF content that user has asked for like this.
                Monthly Budget Plan: 50000
                Essentials: 20000 (Rationale: Covers......
        If none of the above, provide the final budget plan in the requested
Markdown format.

        (Return ONLY the Markdown object)
        ---

        """
```

4.3 Risk Profiling Prompt

```
"""Calculate investment risk profile"""

        prompt = f"""
        You are a certified SEBI-compliant, meticulous investment advisor.
Your SOLE task is to **CALCULATE** the risk profile and return a single,
STRICT, complete Markdown object. **DO NOT perform an analysis or commentary
on missing information.**
```

```
        CHAT HISTORY:
        {self._load_chat_history("default_user")}


        Markdown MUST contain the following fields:
        - risk_level: MUST be one of "Conservative" | "Moderate" |
"Aggressive"
        - risk_score: MUST be an integer between 0 and 100
        - asset_allocation: MUST contain {{Stocks:%, Bonds:%, Cash:%,
RealEstate:%, Alternative:%}}. ALL percentages MUST sum to exactly 100%.
        - rebalancing_schedule: MUST suggest a practical schedule (e.g.,
"Annually", "Semi-Annually").
        - warnings: MUST include 1-2 safety notes (e.g., "Returns are not
guaranteed," "Subject to inflation risk").
        - brief_reason: MUST be a concise, 2-line explanation of the
assessment, *based only on the data provided*.

        STRICT RULES:
        1.  **NO EXTERNAL TEXT:** You MUST NOT output any introductory text,
closing remarks, or explanation outside of the final, complete Markdown
object.
        2.  **IMMEDIATE CALCULATION:** Process the questionnaire data
immediately and jump straight to the Markdown output. Do not analyze missing
fields.
        3.  **DATA INTEGRITY:** Ensure all numerical rules (100% allocation
sum) are met.
        4. **OUTPUT STRUCTURE:** If User has asked about sending email, do not
say that as AI I can not send email. Just provide the dict output with
email_body and attachment_context that user has asked for.
        if User has asked for PDF generation, do not say that as AI I can not
generate PDF. Just provide the Markdown output with PDF content that user has
asked for.

        USER QUESTIONNAIRE (Use this data for calculation):
        {query}

        QUESTIONNAIRE DATA:
        {json.dumps(questionnaire, indent=2)}

        ---
        FINAL RESPONSE:

        If User has asked about sending email or generate PDF, do not say that
as AI I can not send email.
            - do not draft any email content. Just provide the Markdown output
with PDF content that user has asked for.
            - do not mention like this:
```

```
                Subject: Your Personalized Monthly Budget Plan
                Hi Karan,
                Here is the detailed monthly budget plan you requested,
designed to help
                you manage your 50000 income effectively and achieve your
financial goals.

            - ONLY draft PDF content that user has asked for like this.
                Monthly Budget Plan: 50000
                Essentials: 20000 (Rationale: Covers......

        If none of the above, provide the risk assessment in the requested
Markdown format.
        (Return ONLY the Markdown object)

        """
```

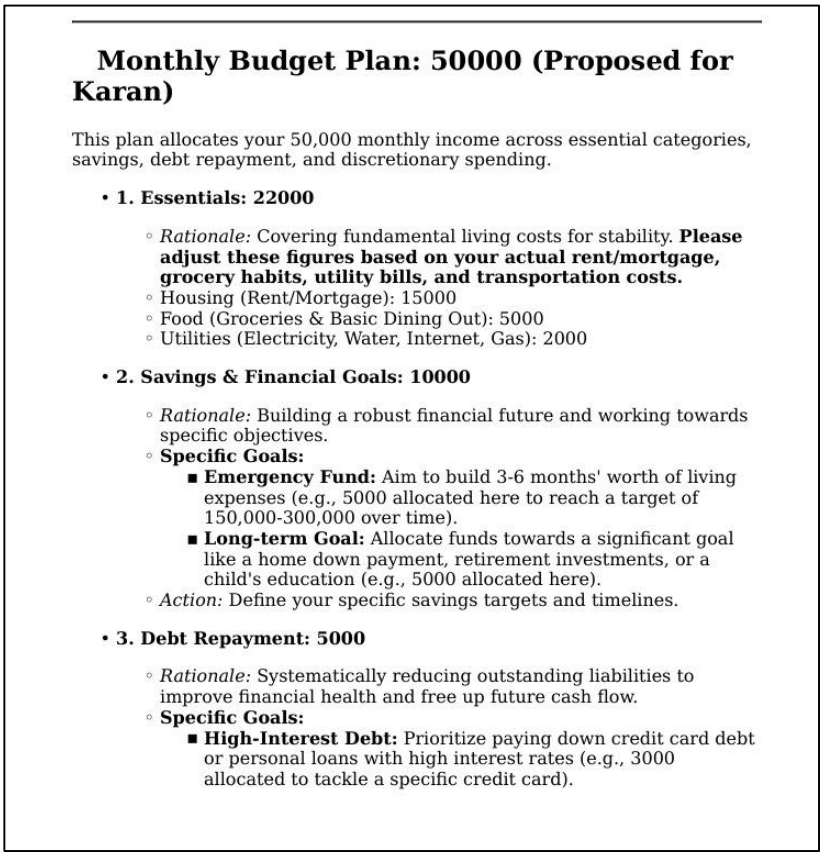
5. Evaluation and Metrics

Metric	Overall	budget_planning	general_qa	risk_profiling
Precision	87.15%	77.78%	91.67%	100.00%
Recall	83.33%	93.33%	73.33%	83.33%
F1-Score	85.19%	84.85%	81.48%	90.91%

The financial assistant demonstrates strong performance with an Overall F1-Score of 85.19%, indicating a healthy balance between accuracy and completeness across its tasks. The model is particularly strong in risk_profiling, achieving 100.00% Precision and a high F1-Score of 90.91%, suggesting exceptional reliability in classifying and responding to risk-related queries. In budget_planning, the high Recall (93.33%) indicates the assistant successfully addresses almost all relevant user requests, though its lower Precision (77.78%) suggests some of those responses may contain minor inaccuracies. Conversely, the general_qa category exhibits the highest Precision (91.67%) but the lowest Recall (73.33%), meaning that while the answers provided for general questions are highly accurate, the model misses or fails to retrieve correct information for a noticeable percentage of these queries.

6. Tool Calling

6.1 PDF

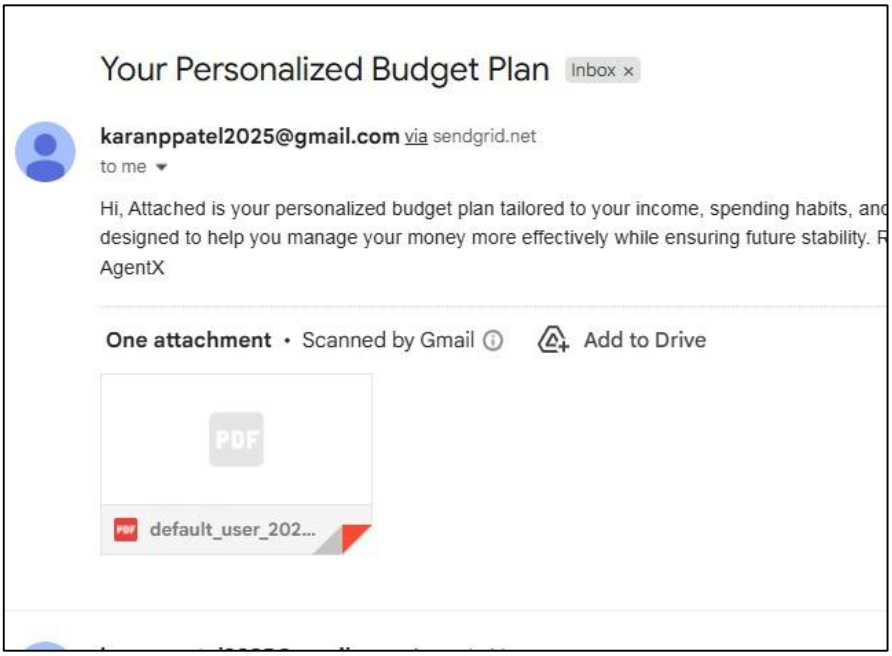


Monthly Budget Plan: 50000 (Proposed for Karan)

This plan allocates your 50,000 monthly income across essential categories, savings, debt repayment, and discretionary spending.

- **1. Essentials: 22000**
  - *Rationale:* Covering fundamental living costs for stability. **Please adjust these figures based on your actual rent/mortgage, grocery habits, utility bills, and transportation costs.**
  - Housing (Rent/Mortgage): 15000
  - Food (Groceries & Basic Dining Out): 5000
  - Utilities (Electricity, Water, Internet, Gas): 2000
- **2. Savings & Financial Goals: 10000**
  - *Rationale:* Building a robust financial future and working towards specific objectives.
  - **Specific Goals:**
    - **Emergency Fund:** Aim to build 3-6 months' worth of living expenses (e.g., 5000 allocated here to reach a target of 150,000-300,000 over time).
    - **Long-term Goal:** Allocate funds towards a significant goal like a home down payment, retirement investments, or a child's education (e.g., 5000 allocated here).
  - *Action:* Define your specific savings targets and timelines.
- **3. Debt Repayment: 5000**
  - *Rationale:* Systematically reducing outstanding liabilities to improve financial health and free up future cash flow.
  - **Specific Goals:**
    - **High-Interest Debt:** Prioritize paying down credit card debt or personal loans with high interest rates (e.g., 3000 allocated to tackle a specific credit card).

6.2 EMAIL



References

1. Using Large Language Models for Financial Advice https://www.researchgate.net/publication/388904444_Using_Large_Language_Models_for_Financial_Advice
2. Biased echoes: Large language models reinforce investment biases and increase portfolio risks of private investors https://portal.fis.tum.de/en/publications/biased-echoes-large-language-models-reinforce-investment-biases-a/
3. Large Language Models for Financial and Investment Management: Applications and Benchmarks https://www.pm-research.com/content/iijpormgmt/51/2/162
4. What is RAG? - Retrieval-Augmented Generation AI Explained https://aws.amazon.com/what-is/retrieval-augmented-generation/
5. How Vanguard worked with Pinecone to boost customer support with faster calls and 12% more accurate responses https://www.pinecone.io/customers/vanguard/
6. Generative AI of Pinecone Vector Retrieval and Retrieval-Augmented Generation Architecture: Financial Data-Driven Intelligent Customer Recommendation System https://www.preprints.org/manuscript/202510.1197
7. Prompt Engineering for AI Guide https://cloud.google.com/discover/what-is-prompt-engineering
8. Prompt Engineering: The Ultimate AI Skill for Investment Operations https://www.empaxis.com/blog/prompt-engineering
9. The Ethics of AI in Financial Planning: Bias, Transparency, and the Role of Human Judgment https://rsisinternational.org/journals/ijrsi/uploads/vol12-iss9-pg01-15-202510_pdf.pdf
10. Ethical Considerations for AI Financial Planning https://www.onestream.com/blog/ethical-considerations-for-ai-financial-planning/

Video link:

https://drive.google.com/drive/folders/1XCM7XDVWNws_7N_KRKn86NzfKhIOr6Lh?usp=sharing